

Lecture 1

course structure
numerical computations in finite precision
round-off errors

course structure and materials

- Course structure:
 - ▶ Lectures: 1 – 2 times a week, 12 lectures, 2 exam-preparation sessions
 - ▶ Practicum (T3): mandatory, pass/fail
 - ▶ Homework (MUMIE), grade 0 – 10
 - ▶ GRASPLE exam, grade 0 – 10
 - ▶ Course grade: if T3='pass', grade=exam(+MUMIE bonus)
 - ▶ Resit (Grasple,T3), no MUMIE bonus
- All information, announcements, and materials: Brightspace
- Teaching team and contacts
 - ▶ Lectures, exam: Neil Budko (n.v.budko@tudelft.nl)
 - ▶ Practicum (T3): Zoë Gromotka (z.j.g.gromotka@tudelft.nl)
 - ▶ Homework: MUMIE support team (mumie-ewi@tudelft.nl)

Course materials

- 1 **Study Guide:** "Werkschema-2026.pdf" – on Brightspace
- 2 **Book:** C. Vuik, F.J. Vermolen, M.B. van Gijzen, M.J. Vuik, *Numerical methods for ordinary differential equations*, Delft Academic Press/VSSD, 2016, reprint 2018 – link to free downloadable copy on Brightspace
- 3 **Calculus book:** J. Stewart, *Calculus: Early Transcendentals*
- 4 **Slides:** Brightspace
- 5 **Recorded lectures:** Brightspace (old)

What is Numerical Mathematics?

By way of definition

Numerical mathematics gives numerical answers to complex mathematical questions. *Numerical analysis* studies numerical approximations (as opposed to symbolic manipulations) for the problems of mathematical analysis.

Computing in finite precision

We know from elementary algebra that

$$\sqrt{4} = 2, \quad \text{because} \quad 2^2 = 4$$

Computer (e.g. Python and Numpy) tells us:

```
In [1]: import numpy as np
```

```
In [2]: np.sqrt(4)
```

```
Out[2]: 2.0
```

We do not know what the answer is to:

$$\sqrt{3} = ?$$

Computer says:

```
In [3]: np.sqrt(3)
```

```
Out[3]: 1.7320508075688772
```

Before we believe it, let's check this answer:

```
In [4]: 1.7320508075688772**2  
Out[4]: 2.9999999999999996
```

Well, maybe computer considers it to be equal 3? Let's check this:

```
In [5]: 3.0 == 2.9999999999999996  
Out[5]: False  
  
In [6]: 3.0 == 2.9999999999999999  
Out[6]: True
```

Computer gives us a wrong answer and it knows it!

How big is the error?

```
In [7]: 3-2.9999999999999996  
Out[7]: 4.440892098500626e-16
```

This is twice the magic number (machine precision, machine epsilon):

```
In [8]: np.finfo(float).eps  
Out[8]: 2.220446049250313e-16  
  
In [9]: 2*np.finfo(float).eps  
Out[9]: 4.440892098500626e-16
```

floating-point numbers and binary representation

floating-point numbers

The floating-point representation of the real number $x \in \mathbb{R}$ in a general number system is defined as

$$\begin{aligned} \text{fl}_\beta(x) &= \pm \left(d_0\beta^E + d_1\beta^{E-1} + d_2\beta^{E-2} + \dots + d_{p-1}\beta^{E-(p-1)} \right) \\ &= \pm (d_0.d_1d_2\dots d_{p-1})_\beta \times \beta^E, \end{aligned}$$

where:

- β is the *base*
- E is the *exponent*
- $(d_0.d_1d_2\dots d_{p-1})_\beta$ is the *mantissa* of x in base β
- $0 \leq d_i \leq \beta - 1$ are integers (*digits*)
- p is the *precision*

example: decimal system

In the *decimal* system, the base is $\beta = 10$. Hence, the number x is expressed as:

$$\begin{aligned} \text{fl}_{10}(x) &= \pm \left(d_0 10^E + d_1 10^{E-1} + d_2 10^{E-2} + \dots + d_{p-1} 10^{E-(p-1)} \right) \\ &= \pm (d_0.d_1 d_2 \dots d_{p-1})_{\beta} \times 10^E. \end{aligned}$$

The mantissa numbers can be the integers $0 \leq d_i \leq 9$.

Example with $E = 0$ and $p = 4$:

$$\begin{aligned} 1.234 &= 1 \times 10^0 + 2 \times 10^{-1} + 3 \times 10^{-2} + 4 \times 10^{-3} \\ &= 1 + 0.2 + 0.03 + 0.004 = 1.234 \times 10^0 \end{aligned}$$

and with $E = 2$ and $p = 4$:

$$\begin{aligned} 123.4 &= 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} \\ &= 100 + 20 + 3 + 0.4 = 1.234 \times 10^2 \end{aligned}$$

Questions: what is the base β and the possible mantissa numbers d_i in the *binary* system? Why computers use binary system?

binary system: storing integers

In the *binary* system the base is $\beta = 2$ and the mantissa numbers are $0 \leq d_i \leq 1$, i.e. 0 or 1. To store an integer J in the $p = 53$ precision (as 53-bit binary), we need to find the 53 mantissa numbers $d_0, \dots, d_{52} \in \{0, 1\}$ and the exponent E such that:

$$J = \pm (d_0 2^E + d_1 2^{E-1} + d_2 2^{E-2} + \dots + d_{52} 2^{E-52})$$

- All integers $|J| < 2^{53}$ can be stored exactly. The largest in this sequence is:

$$J = 1 \cdot 2^{52} + 1 \cdot 2^{51} + \dots + 1 \cdot 2^0 = 2^{53} - 1.$$

- Some integers $|J| \geq 2^{53}$ can also be stored exactly, e.g.

$$J = 2^{53} = 1 \cdot 2^{53} + 0 \cdot 2^{52} + \dots + 0 \cdot 2^1.$$

- Many integers $|J| \geq 2^{53}$ can not be stored exactly, e.g.

$$J = 2^{53} + 1 \sim 1 \cdot 2^{53} + 0 \cdot 2^{52} + \dots + 1 \cdot 2^1 = 2^{53} + 2.$$

storing rational numbers (fractions)

Consider the number $x = 1/10$. In the decimal system with $\beta = 10$ and $E = 0$ this number can be stored with a very short mantissa ($p = 2$):

$$(1/10)_{\beta=10} = 0 \times 10^0 + 1 \times 10^{0-1} + 0 \times 10^{0-2} + \dots = 0.1 \times 10^0$$

However, in the binary system with $\beta = 2$ and $E = 0$:

$$(1/10)_{\beta=2} = (0.0001100110011001100110011\dots)_{\beta=2} \times 2^0 \\ 0 \times 2^0 + 0 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} + \dots$$

it has an infinite mantissa, i.e. needs infinite precision ($p = \infty$).

Therefore, in computers, where $\beta = 2$ and $p = 53$, numbers like 0.1 are often *rounded-off* and only an approximation of 0.1 is stored:

```
In [10]: from decimal import Decimal

In [11]: Decimal.from_float(0.1)

Out[11]: Decimal('0.1000000000000000055511151231257827021181583404541015625')
```

round-off in IEEE-754

In the IEEE-754 standard of the floating point arithmetic, $\beta = 2$ and there is a choice of precisions:

- *single* precision, $p = 24$
- *double* precision, $p = 53$

The double precision floating-point representation of $x = 1/10$ is an approximation of x by a binary number with $p = 53$. It is obtained by looking for the integer N and a 53-bit integer J , such that

$$\frac{1}{10} \sim \frac{J}{2^N}, \quad 2^{52} \leq J \leq 2^{53} - 1$$
$$2^{52} \leq \frac{2^N}{10} \leq 2^{53} - 1$$

round-off in IEEE-754

$$2^{52} \leq \frac{2^N}{10} \leq 2^{53} - 1$$

Looking for N :

```
In [12]: 2**52 <= np.int(2**55/10) <= 2**53-1
Out[12]: False
In [13]: 2**52 <= np.int(2**56/10) <= 2**53-1
Out[13]: True
In [14]: 2**52 <= np.int(2**57/10) <= 2**53-1
Out[14]: False
In [15]: np.int(2**56/10)
Out[15]: 7205759403792794
```

round-off in IEEE-754

Hence, $N = 56$ and $J = 7205759403792794$:

$$\frac{1}{10} \sim \text{fl} \left(\frac{1}{10} \right) = \frac{J}{2^N} = \frac{7205759403792794}{2^{56}} \\ \approx 0.1000000000000000055511151231257827021181583404541015625$$

The rounded result 0.100000000000000001 is correct in the first 16 digits of the mantissa.

finite-precision approximation errors

approximation errors

Let the actual number be

$$x = d_0\beta^{E-0} + d_1\beta^{E-1} + \dots + d_{p-1}\beta^{E-(p-1)} + d_p\beta^{E-p} + \dots$$

and its finite-precision approximation

$$\text{fl}(x) = d_0\beta^{E-0} + d_1\beta^{E-1} + \dots + d_{p-1}\beta^{E-(p-1)}$$

The relative approximation error is

$$\begin{aligned}\frac{\text{fl}(x) - x}{x} &= \frac{d_p\beta^{-p} + d_{p+1}\beta^{-p-1} + \dots}{d_0 + d_1\beta^{-1} + \dots + d_{p-1}\beta^{-p+1} + d_p\beta^{-p} + d_{p+1}\beta^{-p-1} + \dots} \\ &= \beta^{-p} \frac{d_p + d_{p+1}\beta^{-1} + \dots}{d_0 + d_1\beta^{-1} + \dots + d_{p-1}\beta^{-p+1} + d_p\beta^{-p} + d_{p+1}\beta^{-p-1} + \dots}\end{aligned}$$

approximation errors

If $d_p \geq 1$, then:

$$d_p \geq d_{p+1}\beta^{-1} + d_{p+1}\beta^{-2} + \dots$$

This is because:

$$\begin{aligned} d_{p+1}\beta^{-1} + d_{p+1}\beta^{-2} + \dots &\leq (\beta - 1)\beta^{-1} + (\beta - 1)\beta^{-2} + \dots \\ &= (\beta - 1) \sum_{n=1}^{\infty} \frac{1}{\beta^n} = \frac{\beta - 1}{\beta - 1} = 1. \end{aligned}$$

Without loss of generality, we assume that $d_0, d_p \geq 1$. Then

$$\begin{aligned} \frac{\text{fl}(x) - x}{x} &= \beta^{-p} \frac{d_p + d_{p+1}\beta^{-1} + \dots}{d_0 + d_1\beta^{-1} + \dots + d_{p-1}\beta^{-p+1} + d_p\beta^{-p} + d_{p+1}\beta^{-p-1} + \dots} \\ &\leq \beta^{-p} \frac{2d_p}{d_0} \leq 2^{-(p-1)}, \end{aligned}$$

where in the last step we have assumed that $\beta = 2$ and $d_0 = 1, d_p = 1$.

approximation errors and machine epsilon

Absolute Approximation Error = $\text{fl}(x) - x = x\epsilon$

Relative Approximation Error = $\frac{\text{fl}(x) - x}{x} = \epsilon, \quad |\epsilon| \leq \epsilon_m$

$$\boxed{\text{fl}(x) = x(1 + \epsilon)}$$

For example,

$$\begin{aligned} |\epsilon| &= \frac{|0.1 - \text{fl}(0.1)|}{|0.1|} = 10 \left| \frac{1}{10} - \frac{7205759403792794}{2^{56}} \right| \\ &\leq |1 - 1.00000000000000006| = 0.00000000000000006 \\ &= 0.6 \times 10^{-16} \leq \epsilon_m = \frac{1}{2^{52}} = 2.220446049250313\text{e-}16 \end{aligned}$$

Sometimes, a sharper upper bound $|\epsilon| \leq \epsilon_m/2$ is used.

finite-precision floating-point arithmetic

floating point arithmetic

Arithmetic operations, $x + y$, $x - y$, xy , x/y , will have :

- *data* error, due to the floating point representation of the input data
 $x \approx \text{fl}(x)$, $y \approx \text{fl}(y)$
- *conversion* error, due to the conversion of the result to the floating point number, e.g., $x + y \approx \text{fl}(x + y)$. The operations themselves are usually carried out exactly and do not introduce any additional errors. Hence, $\text{fl}(x + y) = \text{fl}(\text{fl}(x) + \text{fl}(y))$.
- connection between $\text{fl}(x)$ and x is: $\text{fl}(x) = x(1 + \epsilon)$

Example: computing x^2

Example: what is the error when we compute x^2 in floating-point arithmetic?

$$\begin{aligned}\text{fl}(x^2) &= \text{fl}(\text{fl}(x) \text{fl}(x)) = \text{fl}(x(1 + \epsilon_d) x(1 + \epsilon_d)) = x^2(1 + \epsilon_d)^2(1 + \epsilon_c) \\ &= x^2(1 + 2\epsilon_d + \epsilon_c + \epsilon_d^2 + 2\epsilon_d\epsilon_c + \epsilon_d^2\epsilon_c) \approx x^2(1 + 2\epsilon_d + \epsilon_c)\end{aligned}$$

For $x = \sqrt{3}$, the absolute error is

$$|\text{fl}((\sqrt{3})^2) - \text{fl}(3)| \approx |3(1 + 2\epsilon_d + \epsilon_c) - 3| = 6\epsilon_d + 3\epsilon_c \leq 9\epsilon_m$$

Or $4.5\epsilon_m$, if we use $\epsilon_m/2$,

```
In [16]: 3-np.sqrt(3)**2  
Out[16]: 4.440892098500626e-16
```

Turns out to be exactly $2\epsilon_m$, but our upper bound $4.5\epsilon_m$ is correct.

propagation and amplification of errors: addition

Suppose we want to compute the sum:

$$\sum_{i=1}^n x_i \quad x_i \in \mathbb{R}.$$

In floating-point arithmetic we get:

$$\begin{aligned}\text{fl}(x_1 + x_2) &= \text{fl}(\text{fl}(x_1) + \text{fl}(x_2)) = (x_1(1 + \epsilon_1) + x_2(1 + \epsilon_2))(1 + \epsilon_{c2}) \\ &= x_1 + x_2 + x_1(\epsilon_1 + \epsilon_{c2}) + x_2(\epsilon_2 + \epsilon_{c2}) + \mathcal{O}(\epsilon^2) \\ \text{fl}(x_1 + x_2 + x_3) &= \text{fl}(\text{fl}(x_1 + x_2) + x_3) = \text{fl}(\text{fl}(x_1 + x_2) + \text{fl}(x_3)) \\ &= x_1 + x_2 + x_3 + x_1(\epsilon_1 + \epsilon_{c2} + \epsilon_{c3}) \\ &\quad + x_2(\epsilon_2 + \epsilon_{c2} + \epsilon_{c3}) + x_3(\epsilon_3 + \epsilon_{c3}) + \mathcal{O}(\epsilon^2) \\ &\dots\end{aligned}$$

propagation and amplification of errors: addition

$$\begin{aligned}\text{fl}\left(\sum_{i=1}^n x_i\right) - \sum_{i=1}^n x_i &= \sum_{i=1}^n x_i \epsilon_i + x_1 \sum_{j=2}^n \epsilon_{cj} + x_2 \sum_{j=2}^n \epsilon_{cj} \\ &\quad + x_3 \sum_{j=3}^n \epsilon_{cj} + x_4 \sum_{j=4}^n \epsilon_{cj} + \cdots + x_n \epsilon_{cn} + \mathcal{O}(\epsilon^2) \\ \left| \text{fl}\left(\sum_{i=1}^n x_i\right) - \sum_{i=1}^n x_i \right| &\leq n \max_i |x_i| \epsilon_m + \frac{1}{2} n(n+1) \max_i |x_i| \epsilon_m + \mathcal{O}(\epsilon^2) \\ &= \left(\frac{1}{2} n^2 + \frac{3}{2} n \right) \max_i |x_i| \epsilon_m + \mathcal{O}(\epsilon^2).\end{aligned}$$

This is an upper bound - worst case scenario. The actual error may be smaller. With the usual rounding strategy we can use $\epsilon_m/2$ instead of ϵ_m , which gives a tighter bound.

amplification of errors in addition: example

Let $x_i = 0.1$ and $n = 10^6$:

$$\sum_{i=1}^{10^6} 0.1 = (0.1)10^6 = 10^5$$

```
In [1]: res = 0
In [2]: for ii in range(int(1e6)):
        ...:     res = res + 0.1

In [3]: res
Out[3]: 100000.00000133288 # this is not equal to 100000
```

The upper bound is

$$\left(\frac{1}{2}(10^6)^2 + \frac{3}{2}10^6 \right) 0.1 \frac{\epsilon_m}{2}, \quad \epsilon_m \approx 2.22 \times 10^{-16}$$

```
In [4]: (0.5*1e6**2+1.5*1e6)*0.1*0.5*2.22e-16
Out[4]: 5.55001665e-06 # (tight) upper bound
In [4]: res-1e5
Out[4]: 1.3328826753422618e-06 # actual error
```

numerically stable algorithms

There are ways to minimize error amplification and accumulation.

```
In [1]: x = np.ones(np.int(1e6))
In [2]: x = x*0.1
In [3]: x
Out[3]: array([0.1, 0.1, 0.1, ..., 0.1, 0.1, 0.1])
In [4]: np.sum(x)
Out[4]: 99999.99999999998
In [5]: np.sum(x)-1e5
Out[5]: -2.0372681319713593e-10
```

The error is much smaller than what we get with the for loop summation!

Question

The `numpy.sum()` function uses the algorithm with pairwise sums of elements. Show that the pairwise summation

$$\text{fl}(\text{fl}(x_1 + x_2) + \text{fl}(x_3 + x_4))$$

results in a smaller floating point error than the sequential addition

$$\text{fl}(\text{fl}(\text{fl}(x_1 + x_2) + x_3) + x_4)$$